Carlos Cotrini

Building ML AI Applications

HNSW: Hierarchical Navigable Small World Graphs

Spring 2026

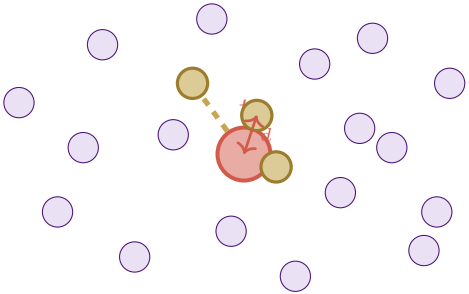
1. Nearest Neighbor Search

Find the K closest points to a query — fast.

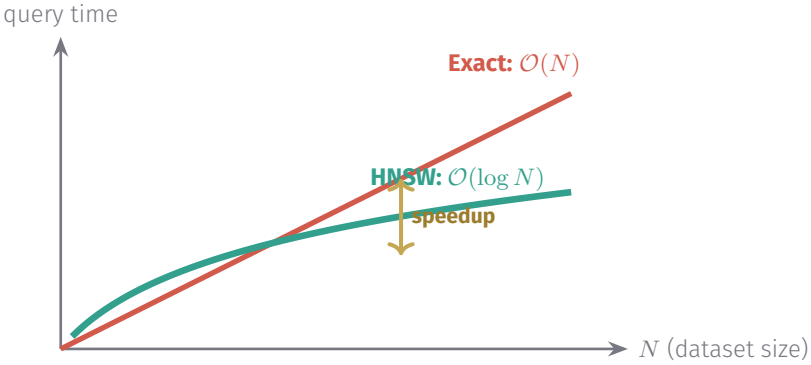
The Task

- N data points
- Query t
- Find K nearest

Exact: scan all N
$\Rightarrow \mathcal{O}(N)$ per query



Goal: Sublinear Search



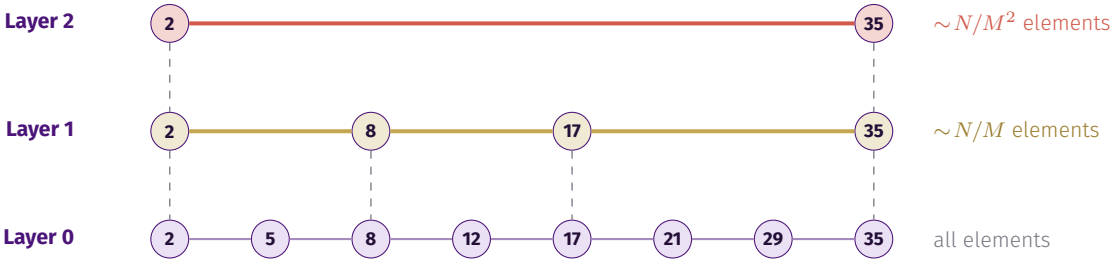
Key idea

Build a **multi-layer graph** that enables **greedy navigation** to nearest neighbors.

2. The Skip-List Analogy

HNSW is a skip list generalized to metric spaces.

Skip Lists: Express Lanes



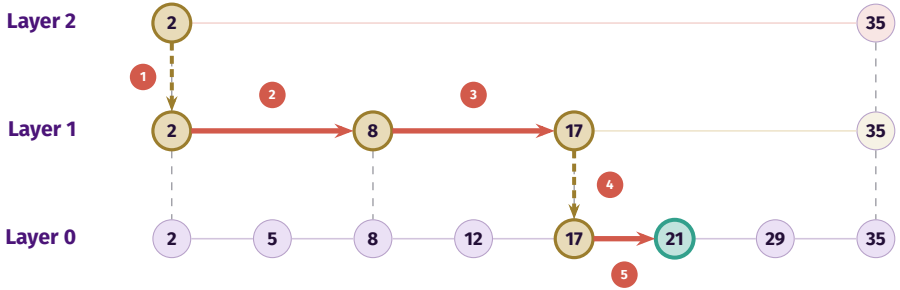
Search strategy:

1. Start at top layer
2. Move right while possible
3. Drop down, repeat

Complexity:

- $\mathcal{O}(\log N)$ expected
- Upper layers = express lanes

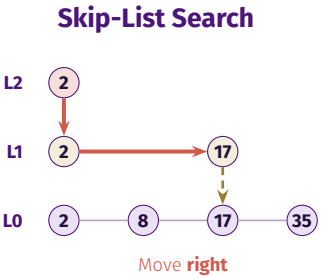
Skip-List Search: Finding 21

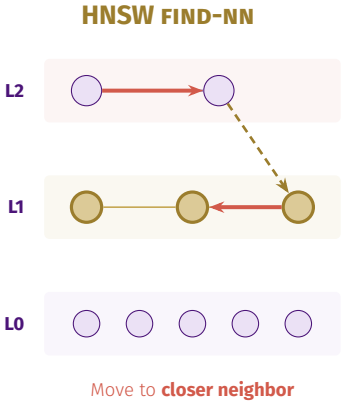


1. **L2, at 2:** next is $35 > 21 \Rightarrow$ drop
2. **L1, at 2:** next is $8 \leq 21 \Rightarrow$ move right
3. **L1, at 8:** next is $17 \leq 21 \Rightarrow$ move right
4. **L1, at 17:** next is $35 > 21 \Rightarrow$ drop
5. **L0, at 17:** next is **21** $\Rightarrow$ found!

5 comparisons vs. 6 for linear scan. Express lanes skip most elements.

Skip-List Search $\longleftrightarrow$ HNSW FIND-NN



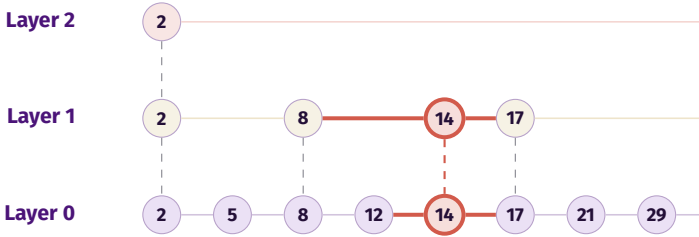


Same strategy: start at top layer, greedily advance, drop when stuck.
Skip list: "next in sorted order" $\longleftrightarrow$ **HNSW:** "closest in metric distance"

Skip-List Insert: Adding 14

Three steps:

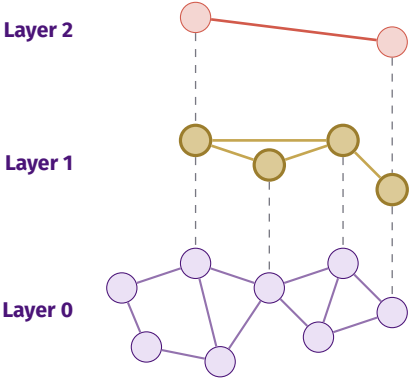
1. **Search** for position
   $\rightarrow$ between 12 and 17
2. **Coin flips** for layer
   H, T $\Rightarrow \ell_{14} = 1$
3. **Splice** into lists
   L0: $12 \rightarrow \mathbf{14} \rightarrow 17$
   L1: $8 \rightarrow \mathbf{14} \rightarrow 17$



Analogy to HNSW INSERT: search $\rightarrow$ BEAM-SEARCH, coin flips $\rightarrow \mathrm{Bern}(1/M)$, splice $\rightarrow$ wire edges via SEL-NEIGHBORS

From 1D to Metric Spaces

Skip List	**HNSW**
Sorted linked list	Proximity graph
Left/right neighbors	M nearest neighbors
Promotion prob. $\frac{1}{2}$	Promotion prob. $\frac{1}{M}$
"Move right"	"Move closer"
Drop to next layer	Drop to next layer



In 1D with $M=2$, HNSW *is* a skip list. In higher dimensions, proximity graphs replace sorted linked lists.

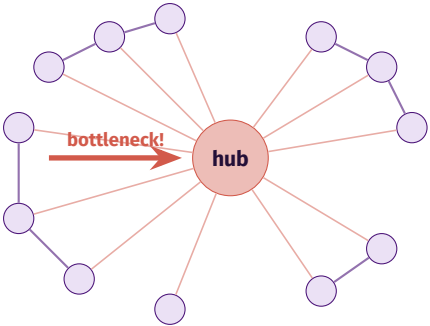
3. Why Layers?

Separating long-range and short-range links.

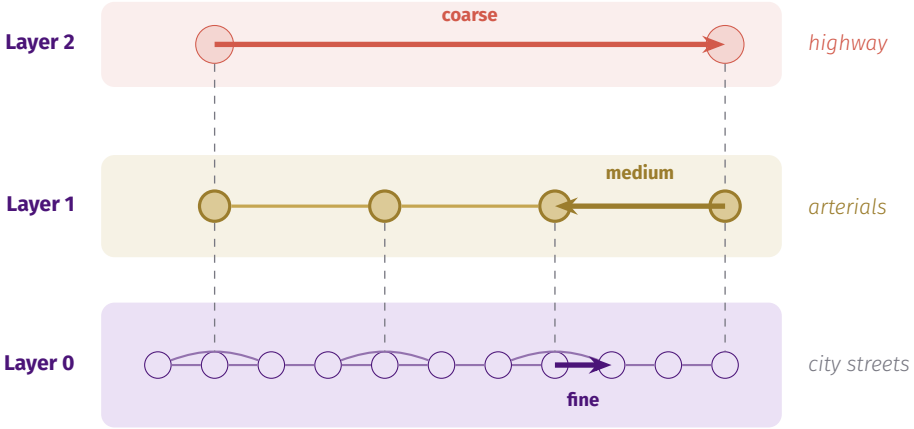
The Hub Problem

Flat graph:

- All links in one layer
- Hub nodes get many connections
- Every search funnels through hubs
- Cost grows with N



The Solution: Layered Hierarchy



Each layer has bounded degree $\leq M$ $\Rightarrow$ no hubs, no bottlenecks.

4. Data Structure

How elements are organized across layers.

Layer Assignment: Coin Flips

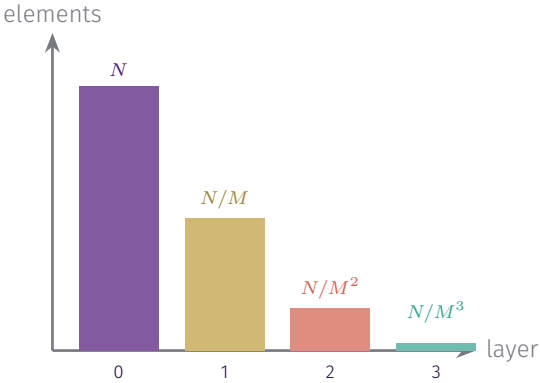
Flip a biased coin (heads prob. $\frac{1}{M}$).

Count heads before first tail:

$$\ell_w = 0, 1, 2, \ldots$$

Geometric distribution:

$$\Pr[\ell_w \geq k] = \left(\frac{1}{M}\right)^k$$



M=16, N=10^6:

Layer 2 ≈ 4000 elements

Typically 4–6 layers total

Key Parameters

Parameter	Typical	Role
M	5–48	Max neighbors per layer
$M_{\max 0} = 2M$	10–96	Max neighbors on layer 0
ef_{Constr}	100–500	Beam width during build
ef	$\geq K$	Beam width at query time

Build-time (fixed)

M and ef_{Constr} are baked into the index. Cannot be changed without rebuilding.

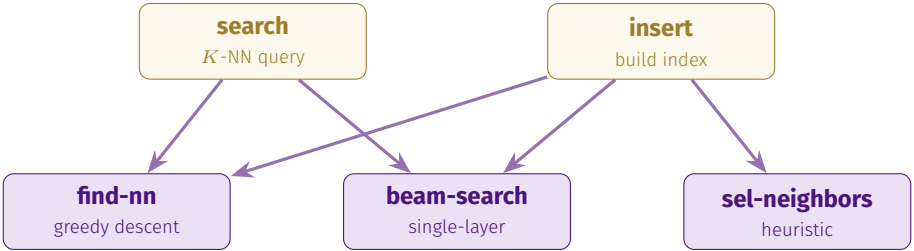
Query-time (tunable)

ef trades speed for recall.
Increase $ef \Rightarrow$ higher recall, slower queries.

5. The Algorithms

Five building blocks of HNSW.

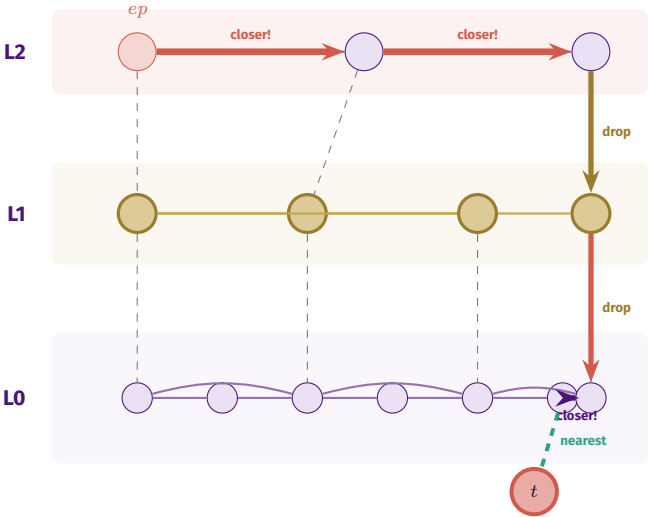
Five Algorithms



FIND-NN: Greedy Descent

1. Start at entry point, top layer
2. Move to closest neighbor
3. No improvement ⇒ drop one layer
4. Repeat until target layer

No beam. Single current node. Fast but can hit local minima.

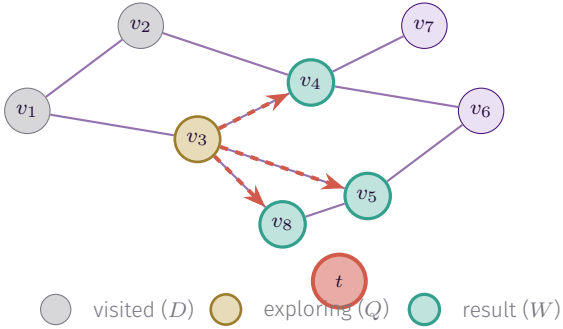


BEAM-SEARCH: Single-Layer Exploration

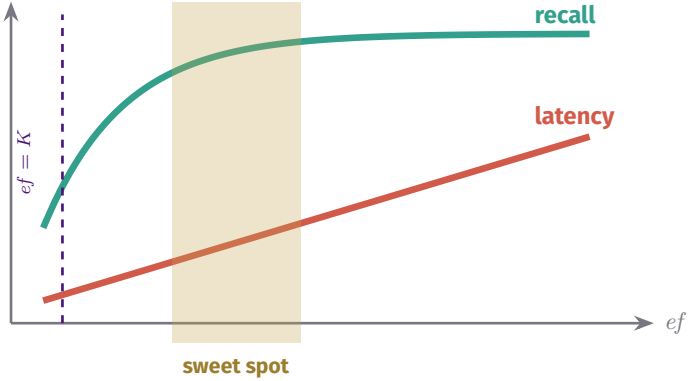
Three data structures:

- Q candidates (min-heap)
- W best ef results (max-heap)
- D visited (hash set)

Stop when best candidate > worst result.



BEAM-SEARCH: The ef Knob



$ef = K$: fastest, lowest recall

$ef = 200$, $K = 10$: examines 200 candidates, returns top 10

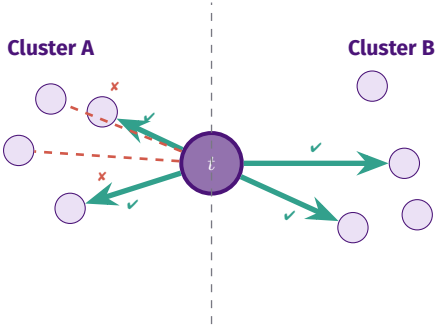
SEL-NEIGHBORS: Directional Diversity

Naïve: pick M closest
$\Rightarrow$ all neighbors in one cluster!

Heuristic: add w^* only if

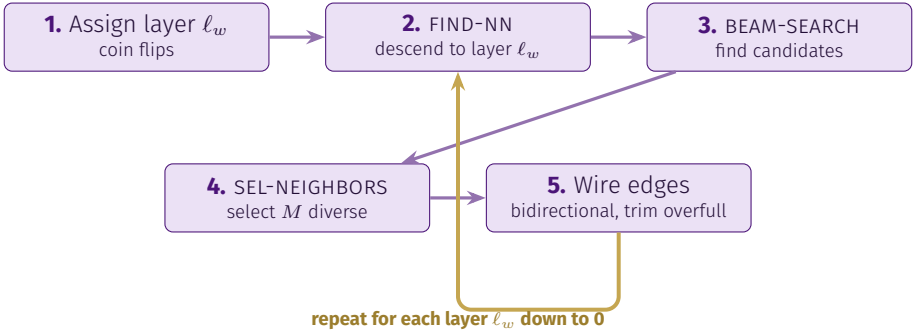
$$d(w^*, t) < \min_{v \in N} d(w^*, v)$$

$\Rightarrow$ angular spread $\approx$ Delaunay



Rejects candidates "overshadowed" by already-chosen neighbors. Ensures connectivity across cluster boundaries.

INSERT: Building the Index



Key detail

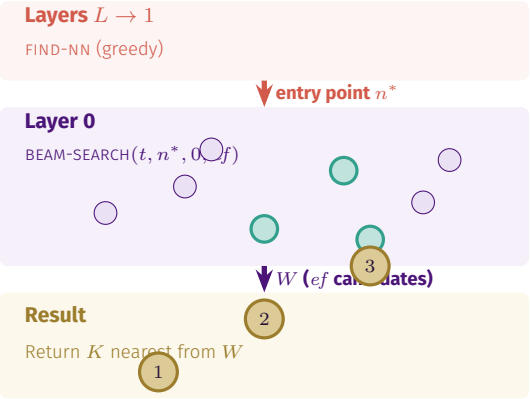
Inserting w may push existing node v over its degree limit.
We trim v's neighbors with SEL-NEIGHBORS — not simple truncation.

SEARCH: Answering K-NN Queries

1. **Greedy descent**
   layers $L \to 1$
   cheap, no beam

2. **Beam search**
   layer 0 only
   find ef candidates

3. **Return top K**



Layer 0 contains *all* N elements — the true nearest neighbor is always there.
Upper layers are just a fast way to find a good starting point.

6. Complexity

Complexity at a Glance

Search

$\mathcal{O}(\log N)$

$\log_M N$ layers

bounded work/layer

+ $ef \cdot M$ on layer 0

Build

$\mathcal{O}(N \log N)$

N insertions

each $\mathcal{O}(\log N)$

+ beam-search/layer

Memory

$\sim 8M$ bytes/elem

$2M$ neighbors (L0)

$+\frac{M}{M-1}$ (upper)

$\approx 2M$ total

Example

$N = 10^6$, $M = 16$: search touches ~ 5 layers $\times$ bounded steps $+ ef \cdot 16$ comparisons on layer 0.
Memory: ~ 128 bytes/element for neighbor indices (plus raw vectors).

7. Worked Example

$M = 4$, inserting points one by one.

Building an Index: $M = 4$

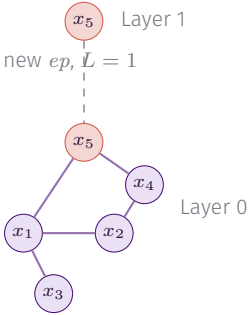
Insert 1

$ep, L = 0$

Insert 2

$\ell_w = 0$

Insert 5: promoted!



Query: SEARCH$(t, ef{=}10, K{=}3)$: start at x_5 on Layer 1, descend, beam-search on Layer 0, return 3 nearest.

8. Practical Tuning

Parameter Guidelines

M

- Default: **16**
- Low-dim: 5–12
- High-dim: 32–64
- More M = more memory

ef_{Constr}

- Default: **200**
- $2\times$ = better graph
- $2\times$ = slower build
- Baked in at build

ef (query)

- Start at $ef = K$
- Increase until recall $\geq$ target
- Free to change per query
- No rebuild needed

Increase ef → *still not enough?* → Rebuild with larger ef_C → *last resort* → Increase M

recall too low?

9. Summary

HNSW in One Slide

Property	Value
Analogy	Skip list
Core idea	Layered links
Search	$\mathcal{O}(\log N)$
Build	$\mathcal{O}(N \log N)$
Memory	$\sim 8M$ B/elem
Key insight	SEL-NEIGHBORS
Knob	ef (query)

Three reinforcing design choices:

1. **Layered hierarchy**
   coarse $\rightarrow$ fine navigation
2. **Heuristic selection**
   connectivity across clusters
3. **Bounded degree**
   no hub bottlenecks

HNSW = skip-list structure + diverse neighbor selection + bounded degree at every layer.

Thank You